

Why Stage 1 Contrastive Training Wasn't Learning (and How Centering Fixed It)

The problem in plain language

The setup: You have audio and text. The model needs to learn that "audio of someone saying *the cat sat*" should match the text *the cat sat*, and shouldn't match *the dog ran*. To compare them, you need to turn each into a single point in a high-dimensional space, then check if matching pairs are close together.

Step 1: What "mean pooling" was doing

Audio and text both arrive as **sequences** of vectors:

- Text "*the cat sat*" → 3 word-vectors, one per word.
- Audio of "*the cat sat*" → maybe 100 little vectors, one per audio chunk.

Contrastive loss works on **single vectors**, not sequences. So you need to compress each sequence into one vector. The simplest way: **average all the vectors together** — that's mean pooling.

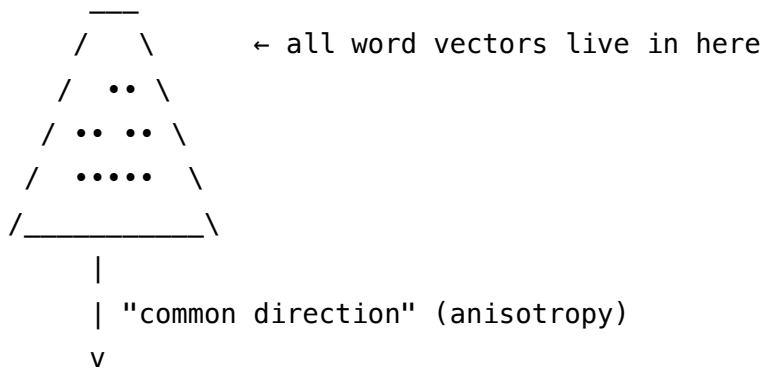
"the cat sat" → $\text{vec}(\text{"the"}) + \text{vec}(\text{"cat"}) + \text{vec}(\text{"sat"}) \div 3$

"the dog ran" → $\text{vec}(\text{"the"}) + \text{vec}(\text{"dog"}) + \text{vec}(\text{"ran"}) \div 3$

Step 2: Why this broke contrastive learning

Here's the catch: **language model embeddings aren't randomly spread out**. When trained, models like Qwen put *all* their word vectors into a narrow region of space — the "common direction" of language.

Imagine if every English word vector lived inside a small cone:



Now when you average 3 vectors that all live in the cone, the average **also lives in the cone** — pretty close to the cone's center. So:

```
mean("the cat sat") ≈ point near cone center
mean("the dog ran") ≈ point near cone center
mean("hello world") ≈ point near cone center
```

All sentences end up looking similar after averaging. That's exactly what our `neg_sim ≈ 0.4` was telling us: random unrelated pairs had a cosine similarity of 0.4 — they were 40% "the same" before training even started.

A concrete analogy

Imagine asking 4 different people for their favorite color, but everyone happens to live in a town where 90% of houses are painted shades of blue. Their answers might be:

- Person A: "navy blue"
- Person B: "sky blue"
- Person C: "teal"
- Person D: "royal blue"

Now ask the model: "*Which person said which color?*" It's nearly impossible — all four answers are *blue-ish*. The differences are real but tiny compared to the giant shared "blueness."

That's what was happening in our loss. The model was being asked: "*Which audio matches which text?*" — but every text-vector was already 40% identical to every other one because of the cone. The signal it needed to learn was drowning in shared "language-ness."

Step 3: Why centering fixed it

The fix is shockingly simple. Before comparing, **subtract out what's common across the batch**:

```
audio_pooled = audio_pooled - audio_pooled.mean(across batch)
text_pooled  = text_pooled  - text_pooled.mean(across batch)
```

Going back to the colors: imagine subtracting the "average blueness" of the batch from each person's answer. Now you're left with only what makes each one *different*:

- Person A: navy – avg_blue = "darker than usual"
- Person B: sky – avg_blue = "lighter than usual"
- Person C: teal – avg_blue = "greener than usual"
- Person D: royal – avg_blue = "more saturated than usual"

Now the model can easily tell them apart. The shared "blueness" was just noise hiding the actual differences.

In geometry terms: we moved every point so the cloud is centered around the origin instead of off in one corner. Now random pairs really do average to near-zero similarity (which is why our `neg_sim` dropped from 0.4 to ~0), and the meaningful pair-by-pair differences became visible.

Why the numbers shifted exactly the way they did

Diagnostic	Before centering	After centering	What it means
text_std	0.008	0.015	The cloud was squashed; centering let it spread out
audio_std	0.012	0.0156	Same — vectors now use the full sphere
neg_sim	0.40	~0	Unrelated pairs are no longer artificially similar
pos_sim	flat at 0.65	climbing 0→0.45	Now there's room to push positives closer
pos_minus_neg	flat at 0.25	climbing 0→0.5	Real contrastive signal, growing each step

Diagnostic	Before centering	After centering	What it means
train/align	stuck at 1.38 (= ln 4)	2.05 → 1.4 and dropping	The loss is now genuinely learning

The general lesson

Contrastive learning doesn't care about *absolute* similarity — it only cares about **the gap between matching pairs and non-matching pairs**. If everything is artificially similar (high baseline), the gap can't grow much. If everything is properly spread out (zero baseline), the gap has room to widen as the model learns.

Mean pooling didn't break the model — it *hid* the model's signal under a thick blanket of "language is language." Centering pulled the blanket off. Same model, same data, same optimizer — just a one-line geometric correction at the loss boundary.

The actual code change

File: training/utils/losses.py

Function: contrastive_infonce_loss

Before:

```
b = audio_tokens.shape[0]
a = F.normalize(audio_tokens.float().mean(dim=1), dim=-1)
t = F.normalize(text_embeddings.float().mean(dim=1), dim=-1)
logits = (a @ t.T) / float(temperature)
return F.cross_entropy(logits, torch.arange(b, device=logits.device))
```

After:

```

b = audio_tokens.shape[0]
a_pooled = audio_tokens.float().mean(dim=1)
t_pooled = text_embeddings.float().mean(dim=1)
a_pooled = a_pooled - a_pooled.mean(dim=0, keepdim=True) # remove common direction
t_pooled = t_pooled - t_pooled.mean(dim=0, keepdim=True)
a = F.normalize(a_pooled, dim=-1)
t = F.normalize(t_pooled, dim=-1)
logits = (a @ t.T) / float(temperature)
return F.cross_entropy(logits, torch.arange(b, device=logits.device))

```

Four added lines. No new parameters. No change to the model, optimizer, or data.

Background — why language model embeddings cluster in a cone

This isn't specific to Qwen. It's a well-documented property of transformer language models:

- **Ethayarajh (2019)** — *How Contextual are Contextualized Word Representations?* First clearly showed that BERT/GPT embeddings are highly anisotropic.
- **Gao et al. (2021)** — *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. Showed that contrastive training itself spreads embeddings out, and that without correction, mean-pooled LM embeddings cluster too tightly to be useful for similarity tasks.
- **Su et al. (2021)** — *Whitening Sentence Representations for Better Semantics and Faster Retrieval*. Proposed full whitening (centering + decorrelation) as a stronger version of the same fix.

The reason it happens: during LM pretraining, the model is rewarded for predicting frequent tokens, which biases the embedding geometry. The high-frequency directions dominate, and rare directions stay underused — producing the cone.

What to watch for in future contrastive runs

If `neg_sim` ever sits well above zero in a contrastive setup, suspect anisotropy first. The fastest checks:

- `Log pooled.std(dim=0).mean()` for each side. Compare to $1/\sqrt{D}$ (≈ 0.0156 for $D=4096$). Much lower means anisotropic.

- Log `pos_sim` and `neg_sim` separately, not just the loss. The loss can decrease for the wrong reason (e.g. stability collapse); the gap cannot.
- If centering isn't enough at scale, the next steps are: (a) projection heads (SimCLR-style MLP), (b) full whitening, or (c) learned attention pooling instead of mean.